

Session 8: Pointers (C Version)

Topics Covered 1. Pointer Theory 2. Pointer Declaration and Dereferencing 3. Pointer Arithmetic 4. Pointers with Arrays and Strings 5. Main Pointer Types in C 6. Activities

1. Pointer Theory

2. **Definition:** A pointer is a variable that stores the memory address of another variable.

3. Advantages:

4. Efficient memory usage.

5. Enables dynamic memory allocation.

6. Allows passing large structures or arrays by reference.

7. Disadvantages:

8. Incorrect use can cause undefined behavior.

9. Can lead to memory leaks or segmentation faults if mismanaged.

10. Pointer Declaration and Dereferencing

11. Declaration:

```
int *p; // pointer to int
char *cptr; // pointer to char
```

12. Assigning Address:

```
int x = 10;
p = &x; // store address of x in pointer
```

13. Dereferencing: Access or modify value at pointer's address

```
printf("%d", *p); // prints 10
*p = 20; // updates x to 20
```

14. Pointer Arithmetic

15. Operations allowed on pointers: +, -, ++, --

```
int arr[] = {1,2,3};
int *p = arr;
printf("%d", *(p+1)); // prints 2
p++; // points to next element
```

16. Pointer arithmetic depends on the size of the data type.

17. Pointers with Arrays and Strings

18. Arrays and pointers are closely related:

```
int arr[3] = {10,20,30};
int *p = arr;
printf("%d", p[1]); // prints 20
```

19. Strings are arrays of characters, so pointers can traverse strings:

```
char str[] = "Hello";
char *ptr = str;
while(*ptr != '\0') { printf("%c", *ptr); ptr++; }
```

20. Main Pointer Types in C

21. **Data Pointers:** Point to variables of specific types (int , char , float *).

22. **Null Pointer:** Points to no valid memory. Value is NULL.

23. **Void Pointer:** Generic pointer; must be cast before dereferencing.

24. **Wild Pointer:** Declared but uninitialized pointer holding random address.

25. **Function Pointer:** Holds address of a function.

26. **Dangling Pointer:** Points to memory that has been deallocated.

27. **Constant Pointers:**

28. **Pointer to Constant:** Data is constant; pointer can be reassigned.

29. **Constant Pointer:** Pointer address is constant; data can be modified.

30. **Constant Pointer to Constant:** Both pointer address and data are constant.

31. Activities **Program: Swap values using pointers**

```
#include <stdio.h>
void swap(int *a, int *b){
    int temp = *a;
    *a = *b;
    *b = temp;
}
int main(){
    int x = 5, y = 10;
    printf("Before swap: x=%d, y=%d\n", x, y);
    swap(&x,&y);
    printf("After swap: x=%d, y=%d\n", x, y);
    return 0;
}
```